

EX NO:1

Basic Javascript Operations

DATE:

Problem Understanding:

The objective is to perform common array and object operations in JavaScript, such as searching, filtering, sorting, transforming, and manipulating data using built-in methods. The program should process the given input and produce the required output efficiently.

1(A)Convert Object to Array of Key-Value Pairs

Given an object, convert it into an array of key-value pairs. Each key-value pair should be in the format [key, value].

```
const user = {  
  name: 'John Doe',  
  age: 30, job:  
  'Developer'  
};
```

CODE:

```
const user = {  
  name: 'John Doe',  
  age: 30, job:  
  'Developer'  
};
```

```
const result = Object.entries(user);  
console.log(result);
```

OUTPUT:

```
PS C:\javascript> node new.js  
24104105 - Esakki Durai M  
[ [ 'name', 'John Doe' ], [ 'age', 30 ], [ 'job', 'Developer' ] ]
```

1(B)Sort an Array of Objects by a Property

An array of objects, each with a name and an age property. Write a function to sort the array in ascending order of age. const people = [

```
{ name: 'John', age: 35 },  
{ name: 'Alice', age: 25 },  
{ name: 'Bob', age: 30 }
```

];

use the sort() method to sort the array by the age property.

CODE:

```
const people = [
```

```
  { name: 'John', age: 35 },
```

```
  { name: 'Alice', age: 25 },
```

```
  { name: 'Bob', age: 30 }
```

```
];
```

```
people.sort((a, b) => a.age - b.age);
```

```
console.log(people);
```

OUTPUT:

```
PS C:\javascript> node new.js  
24104105 - Esakki Durai M  
[  
  { name: 'Alice', age: 25 },  
  { name: 'Bob', age: 30 },  
  { name: 'John', age: 35 }  
]
```

1(C)Remove an Object from an Array by Property

An array of objects, and you need to remove an object based on a specific property value.

```
const people = [ { id: 1, name: 'Alice' },  
  { id: 2, name: 'Bob' },  
  { id: 3, name: 'Charlie' }  
];
```

```
const idToRemove = 2;
```

CODE:

```
const people = [  
  { id: 1, name: 'Alice' },  
  { id: 2, name: 'Bob' },  
  { id: 3, name: 'Charlie' }  
];
```

```
const idToRemove = 2;
```

```
const updatedPeople = people.filter(person => person.id !== idToRemove);
```

```
console.log(updatedPeople);
```

OUTPUT:

```
PS C:\javascript> node new.js  
24104105 - Esakki Durai M  
[ { id: 1, name: 'Alice' }, { id: 3, name: 'Charlie' } ]
```

1(D)Library Book Management

Scenario: You are building a library system where you need to track books. Each book has a title, author, and availability status.

Task:

1. Create an array of books, each with properties: title, author, and available (boolean).
2. Write a function borrowBook(title) that sets the available status of the book to false when a book is borrowed.
3. Write a function returnBook(title) that sets the available status of the book to true when a book is returned.
4. Write a function listAvailableBooks() that lists all books that are available.

CODE:

```
const books = [  
  { title: "Harry Potter", author: "J.K. Rowling", available: true },  
  { title: "The Alchemist", author: "Paulo Coelho", available: true },  
  { title: "Wings of Fire", author: "A.P.J. Abdul Kalam", available: true }  
];
```

```
function borrowBook(title) {  for (let i = 0; i <  
books.length; i++) {      if (books[i].title === title)  
{          if (books[i].available) {  
books[i].available = false;  
console.log(title + " has been borrowed.");  
          } else {  
            console.log(title + " is not available.");  
          }  
        }  
      }  
      console.log("Book not found.");  
    }  
  }
```

```
function returnBook(title) {  for (let i = 0; i <  
books.length; i++) {      if (books[i].title ===  
title) {          books[i].available = true;  
console.log(title + " has been returned.");  
return;  
          }  
        }  
      }  
      console.log("Book not found.");  
    }  
  }
```

```
function listAvailableBooks() { console.log("Available
Books:"); for (let i = 0; i < books.length; i++) { if
(books[i].available) { console.log(books[i].title + "
- " + books[i].author);
    }
  }
}
```

```
listAvailableBooks();
```

```
borrowBook("Harry Potter");
```

```
listAvailableBooks();
```

```
returnBook("Harry Potter");
```

```
listAvailableBooks()
```

OUTPUT:

```
PS C:\javascript> node new.js
24104105 - Esakki Durai M
[ { id: 1, name: 'Alice' }, { id: 3, name: 'Charlie' } ]
PS C:\javascript>
PS C:\javascript> node new.js
24104105 - Esakki Durai M
Available Books:
Harry Potter - J.K. Rowling
The Alchemist - Paulo Coelho
Wings of Fire - A.P.J. Abdul Kalam
Harry Potter has been borrowed.
Available Books:
The Alchemist - Paulo Coelho
Wings of Fire - A.P.J. Abdul Kalam
Harry Potter has been returned.
Available Books:
Harry Potter - J.K. Rowling
The Alchemist - Paulo Coelho
Wings of Fire - A.P.J. Abdul Kalam
PS C:\javascript> █
```

1(E).Movie Rating System

Scenario: You are building a system where users can rate movies from 1 to 5 stars.

- Task:
- 1. Create an object `movies` with the movie names as keys and their ratings (an array) as values.
- 2. Write a function `addRating(movie, rating)` that adds a rating to a movie's list of ratings.
- 3. Write a function `getAverageRating(movie)` that calculates and returns the average rating for a movie.
- 4. Write a function `getHighestRatedMovie()` that returns the movie with the highest average rating.

CODE:

```
const movies = {
  "Leo": [5, 4, 5],
  "Jailer": [4, 5, 4],
  "Vikram": [5, 5, 4]
};

function addRating(movie, rating) {  if
(movies[movie]) {
  movies[movie].push(rating);
  console.log("Rating added to " + movie);
} else {    console.log("Movie
not found.");
}
}

function getAverageRating(movie) {
if (movies[movie]) {    let sum =
0;

    for (let i = 0; i < movies[movie].length; i++) {
sum += movies[movie][i];
    }

    return sum / movies[movie].length;
} else {    return "Movie
not found.";
}
}

function getHighestRatedMovie()
{  let highestMovie = "";  let
highestAverage = 0;
```

```

    for (let movie in movies) {        let
average = getAverageRating(movie);

        if (average > highestAverage) {
highestAverage = average;        highestMovie
= movie;
        }
    }

    return highestMovie;
}

addRating("Leo", 5);

console.log("Average Rating of Leo:", getAverageRating("Leo"));

console.log("HighestRatedMovie:",getHighestRatedMovie());

```

OUTPUT:

```

PS C:\javascript> node new.js
24104105 - Esakki Durai M
Rating added to Leo
Average Rating of Leo: 4.75
HighestRatedMovie: Leo

```

Problem Understand & Approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Viva (5)	Time Management (5)	Total (50)

RESULT:

Successfully created and manipulated JavaScript objects and arrays, and displayed the required data using basic JavaScript operations.